

BUỔI 12

BACKEND:

Models/Order.py

backend > models > order.py > OrderItemDB

```
1  from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey
2  from sqlalchemy.orm import relationship
3  from sqlalchemy.sql import func
4  from database import Base
5
6
7  class OrderDB(Base):
8      __tablename__ = "orders"
9
10     id          = Column(Integer, primary_key=True, index=True)
11     user_id     = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
12     status      = Column(String(20), nullable=False, default="PLACED")
13     total_amount = Column(Float, nullable=False)
14     created_at  = Column(DateTime(timezone=True), server_default=func.now())
15
16     items = relationship("OrderItemDB", back_populates="order", cascade="all, delete-orphan")
17
18
19  class OrderItemDB(Base):
20     __tablename__ = "order_items"
21
22     id          = Column(Integer, primary_key=True, index=True)
23     order_id    = Column(Integer, ForeignKey("orders.id"), nullable=False, index=True)
24     product_id  = Column(Integer, ForeignKey("products.id"), nullable=False)
25     product_name = Column(String(100), nullable=False)
26     product_price = Column(Float, nullable=False)
27     quantity    = Column(Integer, nullable=False)
28     line_total  = Column(Float, nullable=False)
```

Schemas/order.py

```
backend > schemas > order.py > OrderItemQuantityUpdate
1  from pydantic import BaseModel, Field, field_validator
2  from typing import List, Optional
3
4  ALLOWED_STATUSES = ["PLACED", "PROCESSING", "SHIPPED", "COMPLETED", "CANCELED"]
5
6
7  class OrderItemCreate(BaseModel):
8      product_id: int
9      name: str
10     price: float
11     quantity: int = Field(..., gt=0)
12
13
14     class CheckoutRequest(BaseModel):
15         items: List[OrderItemCreate]
16
17
18     class OrderItemRead(BaseModel):
19         id: int
20         product_id: int
21         product_name: str
22         product_price: float
23         quantity: int
24         line_total: float
25
26         class Config:
27             from_attributes = True
28
```

```
class OrderStatusUpdate(BaseModel):
    status: str

    @field_validator("status")
    @classmethod
    def validate_status(cls, v: str) -> str:
        if v not in ALLOWED_STATUSES:
            raise ValueError(f"Invalid status: {v}")
        return v

class OrderItemQuantityUpdate(BaseModel):
    item_id: int
    quantity: int = Field(..., gt=0)
```

routers/orders.py

```
1  from fastapi import APIRouter, Depends, HTTPException, status
2  from sqlalchemy.orm import Session
3  from sqlalchemy import desc
4  from typing import List
5
6  from database import get_db
7  from models.order import OrderDB, OrderItemDB
8  from schemas.order import (
9      CheckoutRequest, OrderRead, OrderItemRead,
10     OrderSummary, OrderStatusUpdate, OrderItemQuantityUpdate,
11 )
12 from auth.deps import get_current_user, require_admin
13
14 router = APIRouter(prefix="/orders", tags=["orders"])
15
16
```

FRONTEND:

OrderHistoryPage.jsx

```
frontend > src > pages > OrderHistoryPage.jsx > default
1  import { useEffect, useState } from 'react';
2  import { useNavigate } from 'react-router-dom';
3  import {
4      Container, Typography, Box, Paper, Table, TableBody,
5      TableCell, TableContainer, TableHead, TableRow, Chip,
6      Button, CircularProgress, Alert,
7  } from '@mui/material';
8  import { Visibility as ViewIcon } from '@mui/icons-material';
9  import { ordersApi } from '../api/ordersApi';
10
11  const statusColors = {
12      PLACED: 'primary',
13      PROCESSING: 'warning',
14      SHIPPED: 'info',
15      COMPLETED: 'success',
16      CANCELED: 'error',
17  };
18
19  const OrderHistoryPage = () => {
20      const navigate = useNavigate();
21      const [orders, setOrders] = useState([]);
22      const [loading, setLoading] = useState(true);
23      const [error, setError] = useState('');
24
25      useEffect(() => {
26          const fetchOrders = async () => {
27              try {
```

OrderDetailPage.jsx

```
frontend > src > pages > OrderDetailPage.jsx > default
1  import { useEffect, useState } from 'react';
2  import { useParams, useNavigate } from 'react-router-dom';
3  import {
4    Container, Typography, Box, Paper, Table, TableBody,
5    TableCell, TableContainer, TableHead, TableRow, Chip,
6    Button, CircularProgress, Alert, Select, MenuItem,
7    FormControl, InputLabel, IconButton, Divider,
8  } from '@mui/material';
9  import {
10    ArrowBack as BackIcon,
11    Add as PlusIcon,
12    Remove as MinusIcon,
13  } from '@mui/icons-material';
14  import { ordersApi } from '../api/ordersApi';
15  import { useAuth } from '../auth/useAuth';
```

AdminOrdersPage.jsx

```
frontend > src > pages > admin > AdminOrdersPage.jsx > default
1  import { useEffect, useState } from 'react';
2  import { useNavigate } from 'react-router-dom';
3  import {
4    Box, Typography, Paper, Table, TableBody, TableCell,
5    TableContainer, TableHead, TableRow, Chip, Button,
6    CircularProgress, Alert,
7  } from '@mui/material';
8  import { Visibility as ViewIcon } from '@mui/icons-material';
9  import { ordersApi } from '../api/ordersApi';
10
11  const statusColors = {
12    PLACED: 'primary', PROCESSING: 'warning',
13    SHIPPED: 'info', COMPLETED: 'success', CANCELED: 'error',
14  };
15
16  const AdminOrdersPage = () => {
17    const navigate = useNavigate();
18    const [orders, setOrders] = useState([]);
19    const [loading, setLoading] = useState(true);
20    const [error, setError] = useState('');
21
22    useEffect(() => {
23      const fetchOrders = async () => {
24        try {
25          const data = await ordersApi.getAllForAdmin();
26          setOrders(data);
27        } catch {
28          setError('Không thể tải danh sách đơn hàng');
29        }
30      };
31    });
32  };
33
34  export default AdminOrdersPage;
```

routes/PrivateRoute.jsx

```
frontend > src > routes > PrivateRoute.jsx > default
1  import { Navigate, Outlet, useLocation } from 'react-router-dom';
2  import { useAuth } from '../auth/useAuth';
3
4  const PrivateRoute = () => {
5    const { isAuthenticated } = useAuth();
6    const location = useLocation();
7
8    if (!isAuthenticated) {
9      return <Navigate to="/login" replace state={{ from: location }} />;
10   }
11
12   return <Outlet />;
13 };
14
15 export default PrivateRoute;
```

Đơn hàng bên khách hàng:

← QUAY LẠI

Đơn hàng #3

22:00:40 12/7/2026

PLACED

Sản phẩm	Đơn giá	Số lượng	Thành tiền
iPhone 15 Pro Max	34.990.000 đ	1	34.990.000 đ

Tổng cộng: 34.990.000 đ

Quản lý đơn hàng:

Quản lý Đơn hàng				
Mã đơn	Trạng thái	Tổng tiền	Ngày đặt	Thao tác
#3	PLACED	34.990.000 đ	22:00:40 12/7/2026	👁️ QUẢN LÝ
#2	PLACED	999 đ	21:48:27 12/7/2026	👁️ QUẢN LÝ
#1	SHIPPED	999 đ	21:17:46 12/7/2026	👁️ QUẢN LÝ